

第十九届全国大学生信息安全竞赛（作品赛）
暨第三届“长城杯”网数智安全大赛（作品赛）

作品报告

☐ 命题赛道

☒ 自由赛道

作品名称： APRA自适应渐进鲁棒聚合
邦学习
电子邮箱： 2833223559@qq.com
提交日期： 2026 年 07 月 04 日

目录

摘要	
41. 作品概述	5
1.1. 背景介绍	5
1.1.1. 联邦学习的兴起与安全挑战	5
1.1.2. 后门攻击的严重性	5
1.2. 应用前景	5
1.2.1. 维护网络空间安全	5
1.2.2. 保障金融数据隐私与模型安全	6
1.2.3. 支撑医疗健康领域的安全协同	6
1.2.4. 赋能自动驾驶与物联网安全	6
1.3. 相关工作	6
1.3.1. 联邦学习后门攻击方法	6
1.3.2. 联邦学习后门防御方法	7
1.4. 作品概述与创新点	7
2. 作品设计	8
2.1. 问题分析	8
2.1.1. 现有防御方法的局限性	9
2.2. 设计思想	9
2.3. 系统架构	11
3. 作品实现	12
3.1. 多维特征提取	12
3.1.1. 参数特征提取	12
3.1.2. NBD/NDIF 行为探针	13
3.1.3. PCA 降维	14
3.2. 自适应 MAD 预过滤	15
3.2.1. MAD 鲁棒统计	15
3.2.2. 自适应阈值衰减	15
3.2.3. 安全保留机制	15
3.3. 层次聚类与可信簇选择	16
3.3.1. 簇数自动选择	16
3.3.2. 可信簇选择	16
3.4. 信任加权与自适应裁剪	17
3.4.1. 信任权重计算	17
3.4.2. 自适应裁剪	18
3.5. 过程追踪与可审计性	19

3.5.1. 客户端级追踪	19
3.5.2. 轮级摘要	19
4. 实验与分析	20
4.1. 实验设置	20
4.2. 主任务准确率	20
4.3. 攻击成功率	21
4.4. APRA 过程筛选分析	23
4.5. 整体对比总结	23
4.6. 各阶段贡献分析	24
5. 创新性分析	26
5.1. 方法创新	26
5.1.1. 多阶段证据融合机制	26
5.1.2. 软降权策略替代硬过滤	26
5.1.3. 自适应阈值调度	26
5.2. 工程创新	27
5.2.1. 统一实验对比框架	27
5.2.2. 全过程可审计追踪	27
5.2.3. Non-IID 场景下的鲁棒性设计	27
6. 总结与展望	27
6.1. 作品总结	27
6.2. 未来展望	28
7. 参考文献	29

摘要

联邦学习允许多个参与方在不共享原始数据的前提下协同训练机器学习模型，在金融、医疗、自动驾驶等隐私敏感领域具有广泛应用前景。然而，联邦学习的分布式特性使其面临严重的后门攻击威胁——攻击者可通过控制部分客户端向全局模型植入后门，使模型在带触发器的输入上输出攻击者指定类别，而主任务准确率几乎不受影响。

本作品提出 APRA (Adaptive Progressive Robust Aggregation, 自适应渐进鲁棒聚合)——一种面向联邦学习后门防御的多阶段鲁棒聚合方法。APRA 的核心思想是将恶意客户端检测问题转化为多维度证据融合与渐进式降权问题：首先通过多维特征提取（参数更新展平、L2 范数、NBD/NDIF 行为探针）将客户端更新转化为可比较的数值特征；随后依次通过自适应 MAD 预过滤、层次聚类与可信簇选择、信任加权与自适应裁剪三个阶段，逐步缩小可疑客户端的聚合贡献。

在 CIFAR-10 和 CIFAR-100 两个数据集上使用 ResNet18 模型的 500 轮联邦训练实验中，APRA 在 A3FL、DOBA、ModelReplace、Neurotoxin、ReBA 五类主流后门攻击下均表现出色：CIFAR-10 上平均主任务准确率 92.13%，平均 ASR 仅 14.10%；CIFAR-100 上平均准确率 65.49%，平均 ASR 仅 1.48%。APRA 在两个数据集上均显著优于 FedAvg、Clip、DeepSight、FoolsGold 和 RFLBAT 等基线方法，尤其在 CIFAR-100 上将平均 ASR 降低了超过 36 个百分点。同时，APRA 内置的全过程审计追踪机制能够记录每轮每个客户端的筛选状态、信任权重和裁剪因子，为防御行为提供可解释性。

作品概述

背景介绍

联邦学习的兴起与安全挑战

联邦学习（Federated Learning, FL）允许多个参与方在不共享原始数据的前提下协同训练机器学习模型。其工作流程为：服务端将当前全局模型分发给被选中的客户端；各客户端使用本地数据训练模型，计算模型权重的变化量（即“模型更新”）；客户端将更新上传至服务端；服务端聚合所有更新，得到新一轮全局模型。由于原始数据始终保留在本地，这一模式天然契合日益严格的数据隐私法规，已在智能手机输入法优化、医疗影像分析、金融风控等场景中得到部署。

然而，联邦学习的分布式架构也打开了新的攻击面。由于服务端无法直接审查客户端的本地数据和训练过程，恶意客户端可以提交精心构造的模型更新，在不显著影响全局模型主任务表现的前提下，向模型中植入后门行为。一个典型的后门攻击场景是：攻击者在本地训练时将特定触发器（如像素模式、水印等）与目标类别关联，使得全局模型在遇到带触发器的输入时稳定输出攻击者指定的错误类别，而在干净样本上表现正常。这种隐蔽性使得后门攻击成为联邦学习面临的最严峻安全威胁之一。

后门攻击的严重性

后门攻击的危害跨越多个关键应用领域。在自动驾驶场景中，被植入后门的模型可能在识别到特定路标图案时将停车标志误分类为限速标志；在医疗诊断系统中，带有特定纹理的医学影像可能被系统性地误诊为目标疾病；在金融风控中，带有特定交易特征的欺诈行为可能被模型系统性放过。这些攻击不仅威胁系统安全，更可能造成人身伤害和重大经济损失。

从技术角度看，联邦学习后门攻击具有三个令人担忧的特征：第一，攻击者只需控制少量客户端即可实现高成功率攻击；第二，攻击可以与正常训练同时进行，服务端难以从准确率指标上察觉异常；第三，持久性攻击（如 Neurotoxin）在攻击停止后仍能使后门长期残留，增加了防御和清除的难度。

应用前景

本作品研究的联邦学习后门防御技术具有广泛的应用前景，主要体现在以下方面：

维护网络空间安全

随着联邦学习在关键信息基础设施中的部署日益增多，后门攻击已成为网络安全防御的新战场。有效的后门防御方法能够保障分布式 AI 系统的模型完整性，防止攻击者通过控制边缘节点向核心模型注入恶意行为，从而维护国家级网络空间安全边界。

保障金融数据隐私与模型安全

金融行业是联邦学习最早也是最重要的应用领域之一。银行、保险等机构需要在跨机构联合建模（如联合风控、反洗钱）中保护客户隐私数据。本作品的方法可以在不访问原始数据的前提下，检测并抑制来自恶意参与方的模型投毒行为，保障联合建模结果的可靠性。

支撑医疗健康领域的安全协同

医疗数据具有极高的隐私敏感性和跨机构协同需求。多家医院联合训练疾病诊断模型时，必须防范恶意参与方植入可能危及患者安全的后门。APRA 的细粒度过程追踪能力在此场景中尤为重要——它可以追溯每一轮训练中每个参与方的行为，为医疗 AI 的安全合规提供审计依据。

赋能自动驾驶与物联网安全

自动驾驶和 IoT 设备通常作为联邦学习的边缘客户端参与训练，这些终端设备更容易被物理攻破或远程控制。APRA 通过多维特征的综合判断，不依赖单一假设来识别恶意更新，对终端安全防护薄弱的边缘场景具有更好的鲁棒性。

相关工作

联邦学习后门攻击方法

近年来，联邦学习后门攻击的手段日趋隐蔽，本作品覆盖了五种代表性攻击：

- 模型替换（Model Replacement）：攻击者首先在本地训练一个带后门的模型，然后将恶意更新的权重放大（乘以一个大于 1 的系数），使其在服务端聚合时能直接覆盖正常客户端的贡献。这种攻击会导致服务端接收到的恶意更新幅度远超正常更新，因此容易被基于更新范数的防御方法检测。
- A3FL：通过在本地训练中引入对抗性优化目标，使生成的恶意更新在参数空间中与正常更新的分布更为接近。具体而言，A3FL 在训练恶意更新时，除了最小化后门任务损失外，还额外约束了恶意更新与预期正常更新之间的方向偏差，从而绕过仅依赖更新幅度或方向检测的防御。
- DOBA：将后门目标分解到多个恶意客户端上协同优化。每个恶意客户端单独提交的更新幅度和方向都与正常更新接近，但这些更新在服务端取平均后，后门信号恰好得到增强。这种分布式协同策略使得单看任何一个客户端的更新都不足以触发告警。
- Neurotoxin：通过分析全局模型参数在正常训练中的更新频率，识别出那些被频繁修改的参数并刻意避开，转而将后门信息写入更新频率较低的区域。攻击停止

后，后续正常训练主要更新高频参数，对低频参数中的后门信息覆盖有限，从而实现了持久性后门植入。

- ReBA: 在触发器设计和训练目标两个层面做了联合优化——自动搜索最优触发器位置和图案，同时调整训练超参数以在保持主任务精度的前提下最大化后门植入效率。

联邦学习后门防御方法

已有防御方法从不同技术角度尝试在服务端识别和抑制恶意客户端更新：

- FedAvg: 标准联邦平均聚合，对每个采样客户端的更新赋予同等权重，不包含任何异常检测机制。在本作品中作为所有防御方法的对比基线。
- Clip: 对每个客户端更新的 L2 范数设置上界，超出上限的更新被等比例缩放到上限值。能有效拦截 ModelReplace 等大幅更新攻击，但无法区分方向正常而幅度合规的恶意更新与真正的良性更新。
- DeepSight: 要求每个客户端提交模型输出层的权重和偏置分布，通过分析客户端模型在干净验证集上的输出分布和参数空间结构，利用聚类 and 异常检测技术识别后门模型。但对 Neurotoxin 等仅修改少量参数的持久性攻击检测效果有限。
- FoolsGold: 记录每个客户端在多轮训练中的历史更新方向，计算客户端之间的余弦相似度矩阵。如果发现多个客户端的更新方向长期高度一致，则降低这些客户端的聚合权重。该方法的假设是：攻击者为植入同一后门会提交方向一致的更新。但在 Non-IID（非独立同分布，即各客户端的数据类别分布不同）场景中，良性客户端因数据不同导致更新方向自然存在差异，容易被误判为低相似度而被降权。
- RFLBAT: 将模型结构分析和鲁棒聚合相结合，对客户端更新进行降维和聚类，通过排除离群簇来降低恶意更新的影响。在部分攻击场景下效果良好，但对攻击策略持续演化的适应性有限。

作品概述与创新点

APRA (Adaptive Progressive Robust Aggregation, 自适应渐进鲁棒聚合) 是本作品提出的联邦学习后门防御方法。它的核心思想是：不追求精确识别每一个恶意客户端，而是通过多阶段渐进式分析，逐步降低可疑客户端对全局模型更新的实际贡献。具体来说，APRA 依次对客户端更新进行多维特征提取、基于鲁棒统计的异常预筛、基于聚类分析的群体判断、以及基于信任权重的贡献控制，将“检测-剔除”的二元范式转化为“粗筛→精选→控权”的渐进式流程。详细的技术设计见第二章。

本作品的创新点体现在四个层面：

- 多阶段证据融合：将鲁棒统计过滤 (MAD)、无监督聚类、相似度加权三种异质信号有机串联。MAD 处理更新幅度的明显异常，聚类处理客户端间的群体关系，信任加权处理个体贡献的精细控制——三个阶段各司其职，避免单一规则的失效导致整体防御崩溃。

- 软降权替代硬过滤：不以二元的“检测并排除恶意客户端”为目标，而是以“降低恶意更新对聚合结果的贡献”为目标。这意味着即使某个恶意客户端未被完全剔除，其更新也会因信任权重较低而被大幅压缩，实际对全局模型的影响远低于其在聚合中的体量占比。
- 自适应阈值调度：MAD 过滤的阈值 $k_t = k_{\text{init}} \exp(-k_{\text{decay}} t)$ 随训练轮次指数衰减。训练早期模型参数变化剧烈，正常更新的方差大，需要宽松阈值避免误筛；训练后期模型趋于收敛，正常更新方向趋同，收紧阈值可更敏感地捕获异常。这种动态调度策略使防御强度与训练阶段自动匹配。
- 全过程可审计追踪：每轮训练不仅输出聚合结果，还记录每个客户端的 MAD 分数、聚类标签、信任权重和裁剪因子，写入结构化 CSV 文件。这使得实验分析可以从最终指标回溯到每一轮、每个客户端的防御行为，回答“防御为什么有效”或“为什么误伤了某个良性客户端”等问题。

作品设计

问题分析

在标准联邦平均算法 FedAvg 中，服务器对所有客户端提交的模型更新做简单算术平均：

$$w_{t+1} = w_t + \frac{1}{n} \sum_{i=1}^n \Delta w_i$$

这个公式背后的假设是：每个客户端都诚实训练，即使个别客户端的更新方向有所偏离，平均后也会被多数正常更新“稀释”。但后门攻击恰恰利用了这种信任——攻击者会刻意构造方向高度一致、精心控制幅度的恶意更新，使得平均之后后门信号非但没被稀释，反而被强化。更棘手的是，攻击者在训练恶意更新时同时约束了主任务损失（即确保模型在正常样本上仍然好用），使得单纯靠“看看模型准确率有没有跌”来检测攻击的方法完全失效——事实上，我们的实验也证实了这一点：即使后门攻击成功率高达 99%，主任务准确率仍然维持在 92% 以上。

本项目关注的五类攻击具有几个共同的隐蔽特征，这些特征使得单纯的幅度检测或准确率监控难以奏效：

1. 幅度伪装：A3FL 和 DOBA 通过分布式协同和自适应优化，使每个恶意客户端单独提交的更新幅度保持在正常范围内，与良性客户端的更新幅度无显著差异，从而绕过基于范数阈值的防御。但当这些小幅更新在服务端被平均后，由于它们指向相同的后门目标方向，叠加效果仍足以在全局模型中植入后门。
2. 持久性设计：Neurotoxin 通过分析模型参数在正常训练中的更新频率分布，识别并避开高频更新的参数区域，将后门信息写入更新频率较低的参数。攻击停止后，

后续正常训练主要修改高频参数，对低频参数中的后门信息覆盖不足，后门得以长期残留。

3. 自适应规避：A3FL 和 ReBA 等攻击在训练过程中根据全局模型状态动态调整攻击策略，包括触发器强度、优化目标和参数修改范围，使得基于固定规则或静态阈值的防御方法难以持续有效。

现有防御方法的局限性

现有五种基线方法各从一个角度尝试防御，但都存在被绕过的途径。下表逐项分析：

- FedAvg：标准联邦平均，不含任何异常检测机制。对于任何被采样到的恶意客户端，其更新在聚合中与良性客户端获得相同的权重，不具备防御能力。
- Clip：对每个客户端更新的 L2 范数（即更新向量所有元素的平方和的平方根，衡量更新幅度的大小）设置上界。当恶意更新幅度被刻意控制在阈值以内时——A3FL 和 DOBA 正是这样做的——裁剪完全无法区分恶意更新和良性更新，因为二者在幅度上没有显著差异。
- FoolsGold：通过计算客户端之间多轮历史更新的余弦相似度来识别协同攻击者。其核心假设是：攻击者为植入同一后门会提交方向一致的更新，因此彼此之间相似度高；良性客户端因数据不同，更新方向自然分散。但这一假设在 Non-IID 场景下会失效——当客户端数据分布本身就不相同时，良性客户端之间的更新方向也存在自然差异，容易导致 FoolsGold 将其误判为可疑并降低其权重。
- DeepSight：要求客户端提交模型输出层的权重和偏置，在服务端对模型输出分布和参数结构进行聚类 and 异常检测。该方法在检测大幅修改的恶意模型时有效，但对 Neurotoxin 等仅修改少量参数的持久性攻击，输出层表征变化微弱，容易漏检。
- RFLBAT：对客户端更新做降维和聚类，通过排除离群簇来抑制恶意更新。在攻击策略固定时表现良好，但面对自适应调整攻击策略的场景，离群簇的定义需要持续更新，否则防御会滞后。

综合来看，每种基线方法都依赖单一维度的信号：Clip 只看幅度、FoolsGold 只看相似度、DeepSight 只看输出层分布。当攻击者的策略恰好规避了该维度的检测时，防御就失效了。这指向 APRA 的设计核心：引入多维特征并通过渐进式流程整合，使各维度互为补充——MAD 处理幅度异常、聚类处理群体关系、信任加权处理贡献控制，从而覆盖更广泛的攻击类型。

设计思想

APRA 的核心假设是：恶意客户端为了植入后门而构造的模型更新，无论伪装得多好，最终都会多个维度上留下可检测的痕迹。具体来说：

- 更新幅度（L2 范数）：恶意客户端如果试图直接替换全局模型（ModelReplace），会产生远远超过正常水平的更新幅度；但 A3FL 和 DOBA 会刻意压制幅度至正常范围以规避检测。

- 输出层偏置：后门攻击需要将带触发器的样本导向目标类别，这会使得分类器输出层中目标类别对应的权重和偏置出现系统性偏移，不同于良性训练的自然分布。
- 噪声输入响应：给模型输入纯随机噪声，正常训练得到模型输出的类别概率接近均匀分布（因为噪声不含任何语义信息）；但如果分类器被植入后门，目标类对应的神经元通道可能被异常强化，导致即使输入噪声，目标类概率也异常偏高。
- 客户端间方向关系：多个恶意客户端为植入同一后门，其更新在参数空间中往往指向相似的方向（高余弦相似度）；而良性客户端因各自持有的 Non-IID 数据不同，更新方向自然分散。

单独的维度可能产生误判——例如 Non-IID 数据本身就会导致良性客户端更新方向存在差异，可能被误认为“方向不一致”——但多维信号合并后，恶意更新与多数正常更新之间的差异会被放大，形成可利用的判定边界。

基于这一思想，APRA 将防御拆解为四个串行阶段，每个阶段的输出是下一阶段的输入：

1. 阶段一——多维特征提取：对每个被采样客户端，提取三类特征：(1) 选定分类器层的展平更新向量，作为方向特征；(2) 整模型更新的 L2 范数，作为幅度特征；(3) 可选地拼接 NBD/NDIF 行为探针——NBD 计算客户端模型与全局模型在输出层偏置上的差异，NDIF 比较两者在随机噪声输入上的输出分布比值。最后通过 PCA 将拼接后的高维向量降维至 3 - 5 维，降低小样本（每轮仅 10 个客户端）下距离度量的噪声。
2. 阶段二——自适应 MAD 预过滤：以所有被采样客户端更新幅度的中位数为中心、MAD (Median Absolute Deviation, 中位绝对偏差) 为尺度，计算每个客户端的修正 z-score。MAD 使用中位数而非均值作为中心估计——均值会被单个极大值拉偏，中位数则保持鲁棒。阈值 $k_t = \max(2.0, k_{\text{init}} \exp(-k_{\text{decay}} t))$ 随轮次 t 指数衰减：训练早期 k_t 接近 k_{init} （默认 5.0），允许较大的更新幅度差异以容纳正常的模型探索；训练后期 k_t 收窄至最小值 2.0，对异常更新更加敏感。若 MAD 过滤后剩余客户端数低于阈值 ($\max(2, \frac{n}{2})$)，触发安全保留机制，保留最接近中位数的客户端，确保训练不会因过度过滤而停滞。
3. 阶段三——层次聚类与可信簇选择：对通过 MAD 的客户端的降维特征向量，使用 Ward 链接的凝聚层次聚类。在 $k \in [2, \max_k]$ 范围内搜索最优簇数，以轮廓系数 (Silhouette Score) 评价每种划分的聚类质量，选择得分最高的 k 。随后对每个簇计算得分：(簇内样本数) $\times$ (簇内平均余弦相似度)，选取得分最高的簇作为可信簇。该策略确保选中“规模大且内部方向一致”的群体，而非简单取最大簇或最近邻。
4. 阶段四——信任加权与自适应裁剪：计算每个可信簇内客户端之间的成对余弦相似度，取每个客户端与其他客户端的最大相似度作为其“代表性得分”，通过 logit 变换映射为信任权重 $w_i \in [0, 1]$ 。信任权重决定了两个关键量：(1) 每个客户端更新在最终聚合中的加权比例；(2) 其裁剪因子 $\text{clip}_i = \text{base_clip} \times \frac{w_i + \epsilon}{\max(w) + \epsilon}$ 。权重

低的客户端不仅聚合占比小，其更新幅度也被更强地压缩，形成“低信任 → 强裁剪 → 低贡献”的双重抑制。

设计取舍

APRA 的设计目标不是输出一个“恶意客户端列表”，而是控制每个客户端更新对全局模型的实际贡献。这种软降权策略比硬过滤有两个优势：(1) 容错——被误判的良性客户端最多被暂时降权，不会被完全剥夺训练参与权；(2) 持续性抑制——未被前两阶段筛选的恶意客户端仍在加权和裁剪阶段被系统性限制贡献，而非一旦漏过就完全不受约束。

系统架构

APRA 的实现遵循模块化设计，核心代码分布在以下文件中：

文件	作用
<code>fl_utils/apra.py</code>	实现 APRA 聚合器，包括特征提取、MAD 过滤、聚类、信任加权聚合和过程记录。
<code>fl_utils/aggregator.py</code>	统一调度 FedAvg、APRA、Clip、DeepSight、FoolsGold、RFLBAT 等聚合方法。
<code>fl_utils/fler.py</code>	联邦训练主循环，负责客户端训练、攻击训练、聚合调用和测试。
<code>fl_utils/attacker.py</code>	实现 A3FL、ReBA、Neurotoxin 等攻击相关训练逻辑。
<code>main/yamls/cifar10_apra.yaml</code>	CIFAR-10 实验配置模板，包含数据集、模型、攻击、防御和 APRA 参数。
<code>main/re_result*</code>	保存训练日志、准确率/ASR 曲线、APRA 客户端筛选记录。

聚合器入口位于 `fl_utils/aggregator.py`。当配置项 `agg_method` 为 `apra` 时，训练循环调用 `APRAAggregator.aggregate`，否则进入相应基线方法。这种设计使 APRA 可以和其他防御在同一训练框架下对比，减少了因训练流程不同造成的实验偏差。

APRA 主流程由以下四个阶段构成：

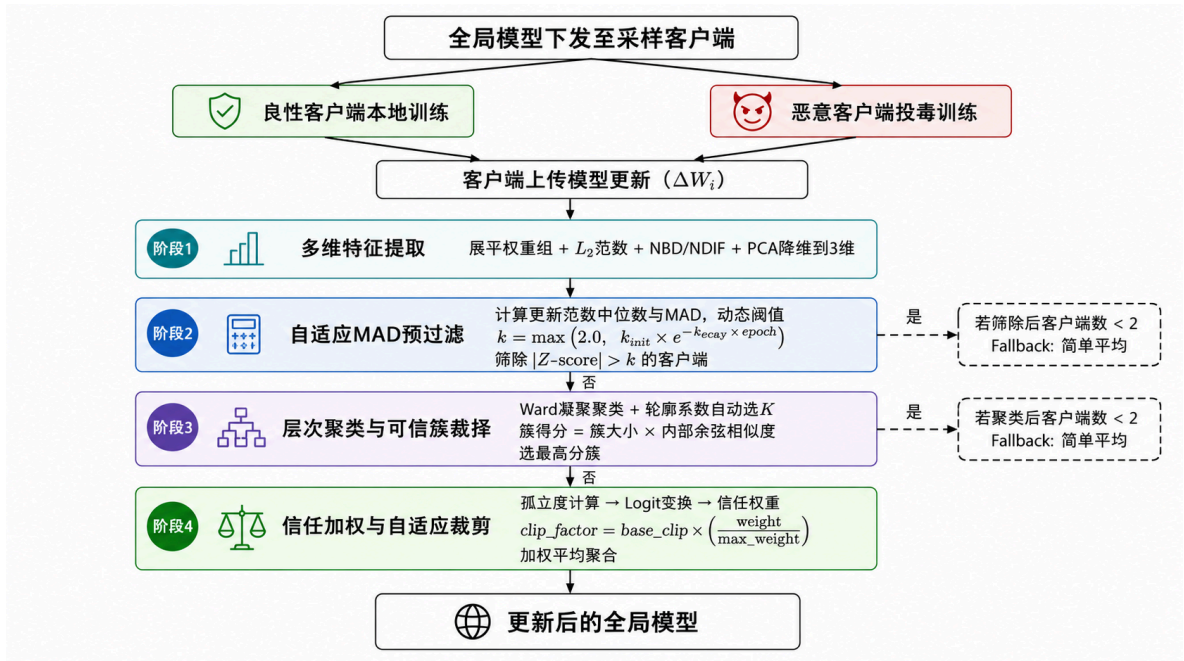


图 1 APRA 四阶段算法流程：特征提取 → MAD 预过滤 → 层次聚类与可信簇选择 → 信任加权与自适应裁剪

四阶段串行执行，前一阶段的输出是后一阶段的输入。每个阶段专门负责一类判断：MAD 处理幅度异常、聚类处理群体关系、信任加权处理个体贡献控制。这种分工使各阶段可以独立调优，也便于通过过程追踪数据分析各阶段的独立贡献。

作品实现

多维特征提取

特征提取是 APRA 的第一阶段，负责将异构的客户端模型更新转化为统一的数值特征向量，供后续阶段使用。

参数特征提取

APRA 首先通过 `flatten_update` 对指定的关键层参数做展平操作。对于 CIFAR-10 数据集，目标层为分类器输出层（`linear`），因为该层最接近类别决策边界，后门攻击的目标类偏置在该层更新中最容易被观察。

```
def flatten_update(update_dict, layer_names=None):
    flat = []
    for name, data in update_dict.items():
        if layer_names and not any(l in name for l in layer_names):
```

```

        continue
    if 'num_batches_tracked' in name:
        continue
    flat.append(data.cpu().numpy().flatten())
return np.concatenate(flat) if flat else np.array([])

```

跳过 `num_batches_tracked` 是因为它是 BatchNorm 的计数统计量，不适合作为连续向量参与范数、PCA 或相似度计算。

同时，APRA 计算每个客户端整体模型更新的 L2 范数，用于后续 MAD 预过滤：

```

l2 = 0.0
for name, data in update.items():
    if 'num_batches_tracked' in name:
        continue
    l2 += torch.norm(data, p=2).item() ** 2
update_norms.append(np.sqrt(l2))

```

设计思路

APRA 将“是否异常”拆成两个视角：范数异常反映更新幅度是否过大或过小（适合检测 ModelReplace 等大幅度攻击），方向特征反映更新是否偏离正常类别边界（适合检测低幅度但方向可疑的攻击，如 A3FL、DOBA）。二者配合能够覆盖更多攻击类型。

NBD/NDIF 行为探针

当配置启用 NBD/NDIF 时，APRA 不仅观察参数差异，还向客户端模型输入随机噪声，观察输出分布是否异常偏向某一类别：

```

rand_input = torch.randn((32, 3, 32, 32)).to(device)
global_output = torch.mean(
    torch.softmax(global_model(rand_input), dim=1), dim=0
) + 1e-8

client_output = torch.mean(
    torch.softmax(client_models[client_id](rand_input), dim=1), dim=0
)
ndif = (client_output / global_output).cpu().detach().numpy()

```

设计思路

NBD (Neuron Bias Difference) 观察输出层偏置变化, NDIF (Neuron Distribution Imbalance Factor) 使用随机噪声探测模型行为盲测。随机噪声不是为了获得语义正确的分类结果, 而是为了放大异常目标类偏好。正常模型对噪声输入通常不会稳定偏向同一类别; 后门模型则可能因为目标类通道被强化而出现异常响应。

PCA 降维

展平特征与 NBD/NDIF 拼接后, 通过 PCA 降维:

```
n_components = min(config['apra_pca_components'], *flat_features.shape)
if flat_features.shape[0] >= 2 and n_components >= 2:
    pca = PCA(n_components=n_components)
    if np.isnan(flat_features).any() or np.isinf(flat_features).any():
        flat_features = np.nan_to_num(
            flat_features, nan=0.0, posinf=1.0, neginf=-1.0
        )
    features = pca.fit_transform(flat_features)
```

降维的主要目的不是可视化, 而是降低小样本聚类的噪声。每轮只采样 10 个客户端, 如果直接在高维参数空间做距离度量, 冗余维度会主导距离计算; PCA 将主要变化压缩到少数维度, 使后续轮廓系数和余弦相似度更稳定。

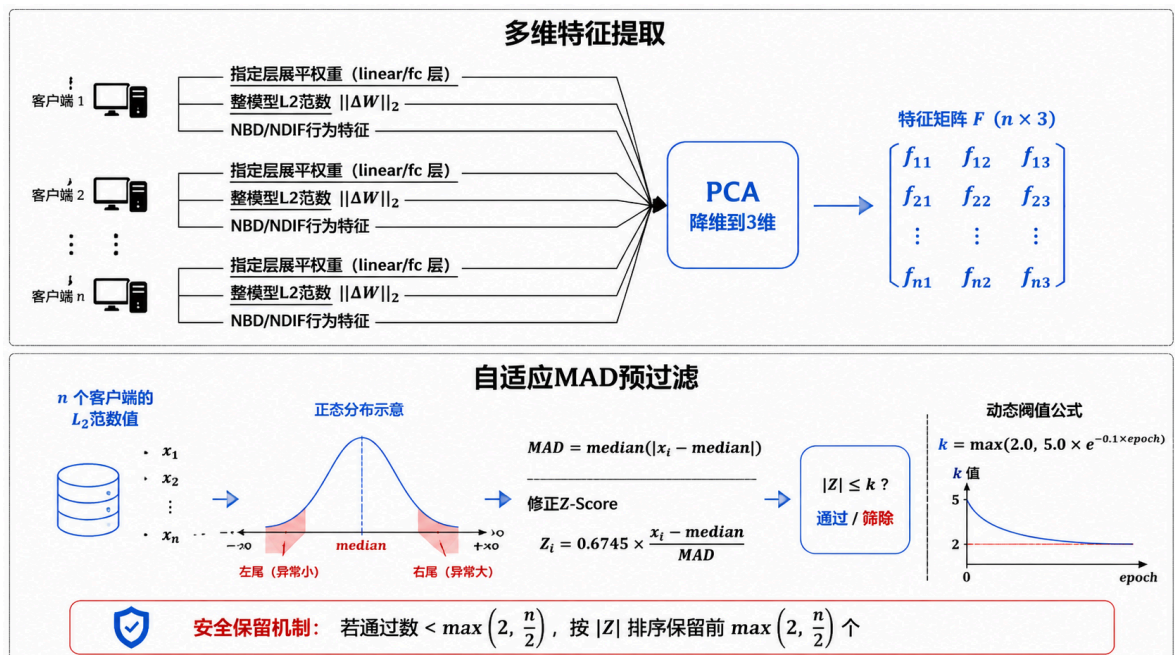


图 2 多维特征提取与 MAD 过滤流程

自适应 MAD 预过滤

MAD 鲁棒统计

MAD 预过滤使用中位数和中位绝对偏差 (Median Absolute Deviation) 替代传统的均值和标准差, 原因是中位数统计量对极端恶意更新不敏感, 不会被少量大范数攻击样本拉偏。

核心计算公式为:

$$z_i = 0.6745 \frac{n_i - \text{median}(n)}{\text{MAD}(n)}$$

$$k_t = \max(2.0, k_{\text{init}} \exp(-k_{\text{decay}} t))$$

其中 k_{init} 和 k_{decay} 来自配置文件 (默认 $k_{\text{init}} = 5.0$, $k_{\text{decay}} = 0.1$)。常数 0.6745 使得修正 z-score 在正态分布假设下近似于标准 z-score。

```
k = max(2.0, k_init * np.exp(-k_decay * epoch))
median_norm = np.median(update_norms)
mad = np.median(np.abs(update_norms - median_norm))

modified_z_scores = 0.6745 * (update_norms - median_norm) / mad
mask = np.abs(modified_z_scores) <= k
```

自适应阈值衰减

阈值 k_t 随训练轮次 t 指数衰减。训练早期 (t 较小), k_t 接近 k_{init} , 阈值宽松, 有利于模型正常收敛, 不会因为过度过滤而损失训练信号。随着训练推进, k_t 逐渐收紧至最小值 2.0, 对后期持续注入的异常更新形成更强抑制。

安全保留机制

如果 MAD 一次性筛掉过多客户端, APRA 会触发安全保留——保留最接近中位数的一半客户端, 避免服务端在本轮几乎没有训练信号可聚合:

```
if mask.sum() < max(2, n_clients // 2):
    sorted_indices = np.argsort(np.abs(modified_z_scores))
    keep_count = max(2, n_clients // 2)
    mask[:] = False
    mask[sorted_indices[:keep_count]] = True
    safety_keep_used = True
```

设计思路

这是一处关键的鲁棒性设计。防御算法不能只追求“剔除可疑客户端”，还必须保证训练过程持续推进。在 Non-IID 场景下，正常客户端的更新范数也可能差异较大；安全保留机制能降低单轮误判导致全局模型停滞或剧烈震荡的风险。

层次聚类与可信簇选择

MAD 过滤后，APRA 使用凝聚层次聚类（Agglomerative Clustering）对剩余客户端进行分组。与 K-Means 等需要预设簇数的方法不同，凝聚层次聚类更适合每轮仅 10 个客户端的小样本设置。

簇数自动选择

APRA 在可行范围内搜索最优簇数，用轮廓系数（Silhouette Score）评价聚类质量：

```
for k in range(2, max_k + 1):
    clustering = AgglomerativeClustering(
        n_clusters=k, metric='euclidean', linkage='ward'
    )
    labels = clustering.fit_predict(features)
    score = silhouette_score(features, labels)
    if score > best_score:
        best_score = score
        best_k = k
```

可信簇选择

选择可信簇时，APRA 综合考虑簇规模和簇内方向一致性，而非简单地选择最大簇：

```
for c in np.unique(labels):
    indices = np.where(labels == c)[0]
    cluster_size = len(indices)
    if cluster_size <= 1:
        cluster_scores[c] = -1
        continue
    internal_sim = np.mean(sk_cosine_similarity(features[indices]))
    cluster_scores[c] = cluster_size * internal_sim

selected_cluster = max(cluster_scores, key=cluster_scores.get)
```

若某个簇人数多但内部方向混乱，其得分会因余弦相似度低而下降；若某个簇只有单个客户端，即使它距离特殊，也不会被视为可信主体。这一策略使 APRA 更偏向选择“多数且方向一致”的客户端群体，符合联邦学习中“多数客户端为良性”的基本假设。

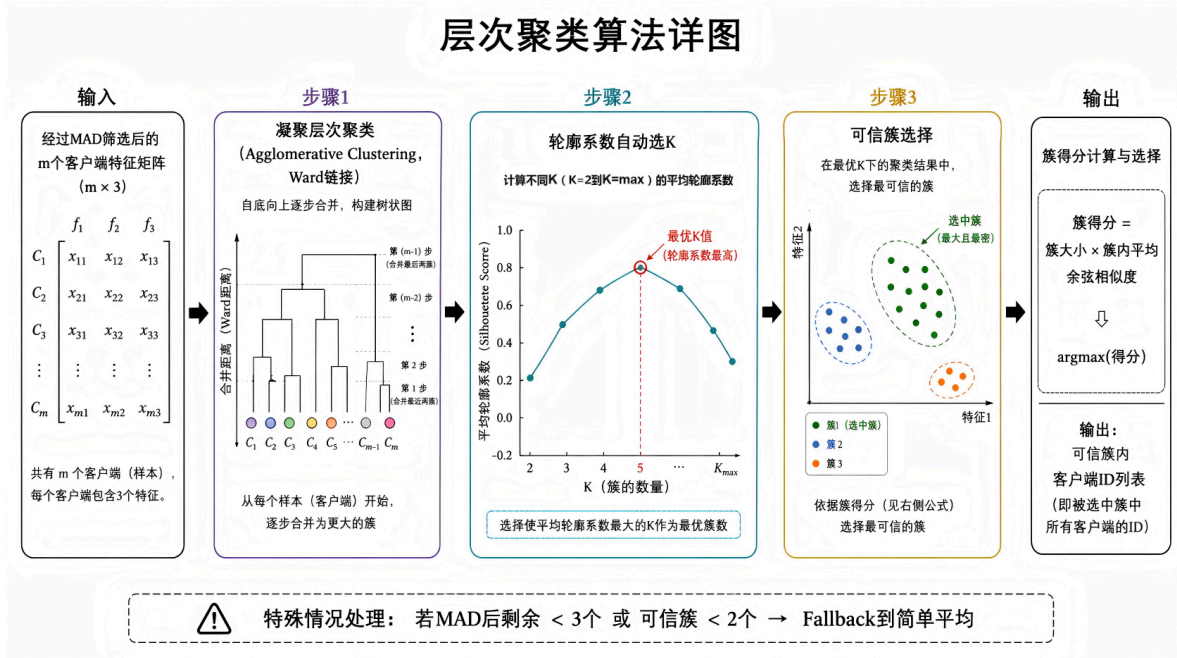


图 3 层次聚类与可信簇选择流程

信任加权与自适应裁剪

通过聚类筛选的客户端并不被完全等同对待。APRA 继续计算保留客户端之间的余弦相似度，通过 logit 变换将其转化为信任权重，并基于信任权重对每个客户端进行自适应裁剪。

信任权重计算

```
cs = sk_cosine_similarity(features)
np.fill_diagonal(cs, -1.0)
maxcs = np.max(cs, axis=1) + epsilon

wv = 1 - np.max(cs, axis=1)
wv = np.clip(wv, 0, 1)
wv = wv / np.max(wv)
wv[wv == 1] = 0.99
wv = np.log(wv / (1 - wv) + epsilon) + 0.5
wv = np.clip(wv, 0, 1)
```

```
wv = 1.0 - wv
wv = wv / np.sum(wv)
```

这段代码的设计思路是从“硬选择”过渡到“软降权”。即使某个客户端通过了 MAD 和聚类，仍然会因为与可信群体方向不够一致而获得较低权重。logit 变换将相似度差异放大，使方向不一致的客户端获得明显更低的权重。

自适应裁剪

信任权重进一步决定每个客户端的裁剪阈值：

```
for i in range(n):
    w_i = trust_weights[i]
    ratio = (w_i + 1e-8) / (max_trust + 1e-8)
    clip_factors[i] = base_clip * ratio

for key in update:
    if 'num_batches_tracked' in key:
        continue

    l2 = torch.norm(update[key], p=2)
    update[key].div_(max(1.0, l2.item() / cf))
```

权重越低的客户端，裁剪因子越小，其更新被压缩得越厉害。聚合阶段不再按客户端数量平均，而是按信任权重累加更新，高信任客户端对全局模型的更新贡献更大。

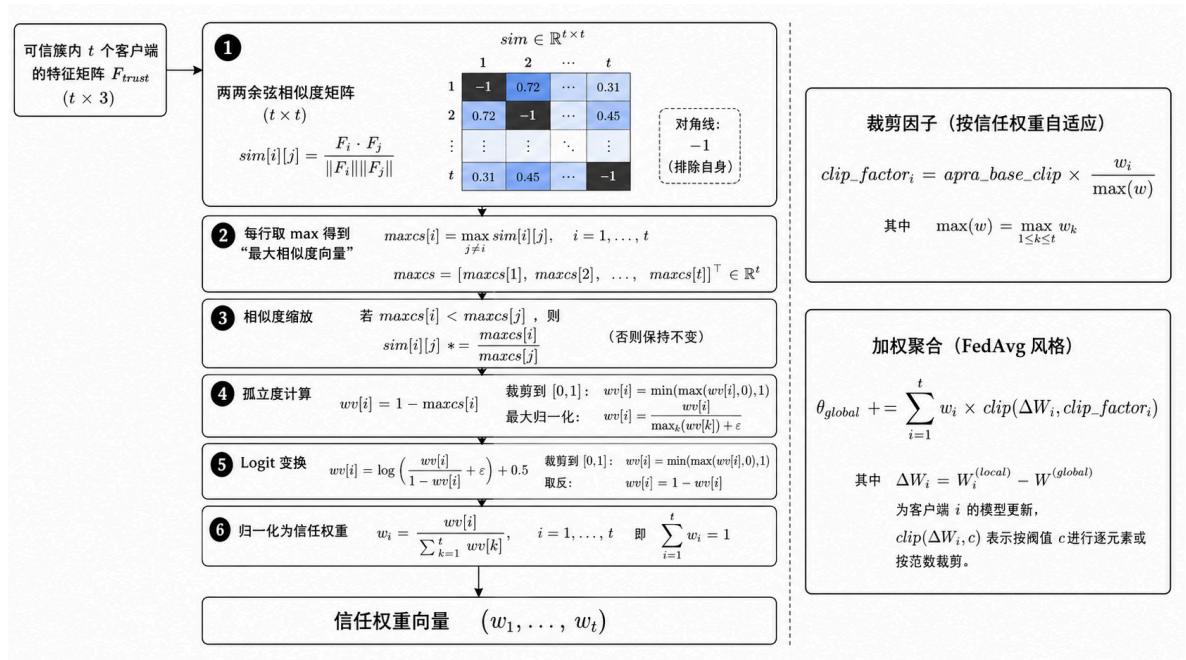


图 4 信任加权计算与自适应裁剪流程

设计思路

APRA 的最终目标不是输出“攻击者名单”，而是控制每个客户端对全局模型的实际影响。MAD、聚类、信任权重和裁剪分别对应“初筛、群体判断、贡献排序、幅度限制”。四步串行后，即使某些恶意客户端没有被完全剔除，它们的更新也被限制在较小贡献内，从而在保证训练可继续的前提下最大化防御效果。

过程追踪与可审计性

APRA 的一个特色功能是将每轮筛选信息写入结构化 CSV 文件，使实验结果具有可解释性和可审计性。

客户端级追踪

`apra_client_trace.csv` 记录每个客户端在每轮训练中的完整状态：

```
rows.append({
    "epoch": int(epoch),
    "client_id": client_id,
    "role": role,
    "update_norm": self._safe_float(update_norms[idx]),
    "mad_z_score": self._safe_float(z_scores[idx]),
    "mad_effective_pass": int(bool(mad_effective_mask[idx])),
    "cluster_label": int(cluster_labels[idx]),
    "final_selected": int(client_id in chosen_set),
    "trust_weight": self._safe_float(trust_by_client.get(client_id, "")),
    "clip_factor": self._safe_float(client_clip_info.get("clip_factor", "")),
})
```

轮级摘要

`apra_round_summary.csv` 提供每轮训练的整体摘要：

```
summary = {
    "epoch": int(epoch),
    "sampled_malicious_ids": self._json_list(malicious_in_sample),
    "mad_reject_malicious_ids": self._json_list(mad_rejected_malicious),
    "cluster_selected_cluster": int(selected_cluster),
    "final_selected_ids": self._json_list(final_selected_ids),
    "final_rejected_ids": self._json_list(final_rejected_ids),
}
```

这些记录使实验分析可以回答更细粒度的问题：某一轮攻击者是否被 MAD 筛掉？是否进入可信簇？最终是否参与聚合？参与后权重和裁剪因子是多少？没有这类记录，就只能看到最终准确率和 ASR，很难解释防御为什么有效或为什么误伤正常客户端。

实验与分析

实验设置

实验在两个标准图像分类数据集上进行：

1. CIFAR-10: 10 类彩色图像，模型为 ResNet18，联邦训练 500 轮，客户端总数 100，每轮采样 10 个客户端，其中 2 或 5 个为恶意客户端（依攻击类型而定）。数据划分为 Non-IID，Dirichlet 分布参数 0.7。
2. CIFAR-100: 100 类彩色图像，模型为 ResNet18，联邦训练 500 轮（ReBA 攻击为 700 轮），其余参数与 CIFAR-10 一致。

实验评估采用以下三个维度的指标：

1. 主任务准确率（Main Task Accuracy）：模型在干净测试集上的分类准确率，反映模型保留正常分类能力的程度，越高越好。
2. 攻击成功率（Attack Success Rate, ASR）：带有后门触发器的测试样本被误分类为攻击目标类别的比例，反映防御对后门行为的抑制效果，越低越好。
3. APRA 过程筛选指标：恶意客户端被剔除比例、良性客户端误剔除比例等，用于解释防御过程的内部行为。

对比方法包括 FedAvg、Clip、DeepSight、FoolsGold、RFLBAT 和 APRA 共六种聚合策略。攻击类型包括 A3FL、DOBA、ModelReplace、Neurotoxin 和 ReBA 共五种主流后门攻击。报告中的汇总数值以训练末轮数据为口径。

主任务准确率

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	92.08	92.39	92.35	92.11	92.33	92.29
DOBA	92.08	92.43	92.41	92.08	92.24	92.21
ModelReplace	92.05	92.45	92.45	92.02	92.35	92.33
Neurotoxin	92.27	92.47	92.41	91.98	92.45	92.28
ReBA	92.18	92.61	92.61	92.21	92.49	92.68
平均	92.13	92.47	92.45	92.08	92.37	92.36

表 1: CIFAR-10 主任务准确率 (%), 500 轮末轮数据。

CIFAR-10 上各方法准确率集中在 92% 附近，差距很小。APRA 平均准确率 92.13%，略低于 FedAvg 的 92.47%，但差距仅 0.34 个百分点，没有出现模型崩溃。

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	65.93	67.71	67.64	65.50	67.79	67.68
DOBA	65.75	67.68	67.79	65.46	67.79	67.54
ModelReplace	65.54	66.43	67.51	65.78	65.97	67.48
Neurotoxin	65.74	67.72	67.77	65.30	67.96	67.64
ReBA	64.50	67.77	67.74	65.51	68.00	67.62
平均	65.49	67.46	67.69	65.51	67.50	67.59

表 2: CIFAR-100 主任务准确率 (%), 500 轮末轮数据 (ReBA 为 700 轮)。

CIFAR-100 上任务本身更难, 各方法准确率在 65% - 68% 区间。APRA 平均准确率 65.49%, 与最佳基线 (Clip 67.69%) 差距在 2.2 个百分点以内。CIFAR-100 上 100 类的分类任务对参数变化更敏感, 但 APRA 仍然保持了可用的主任务能力。

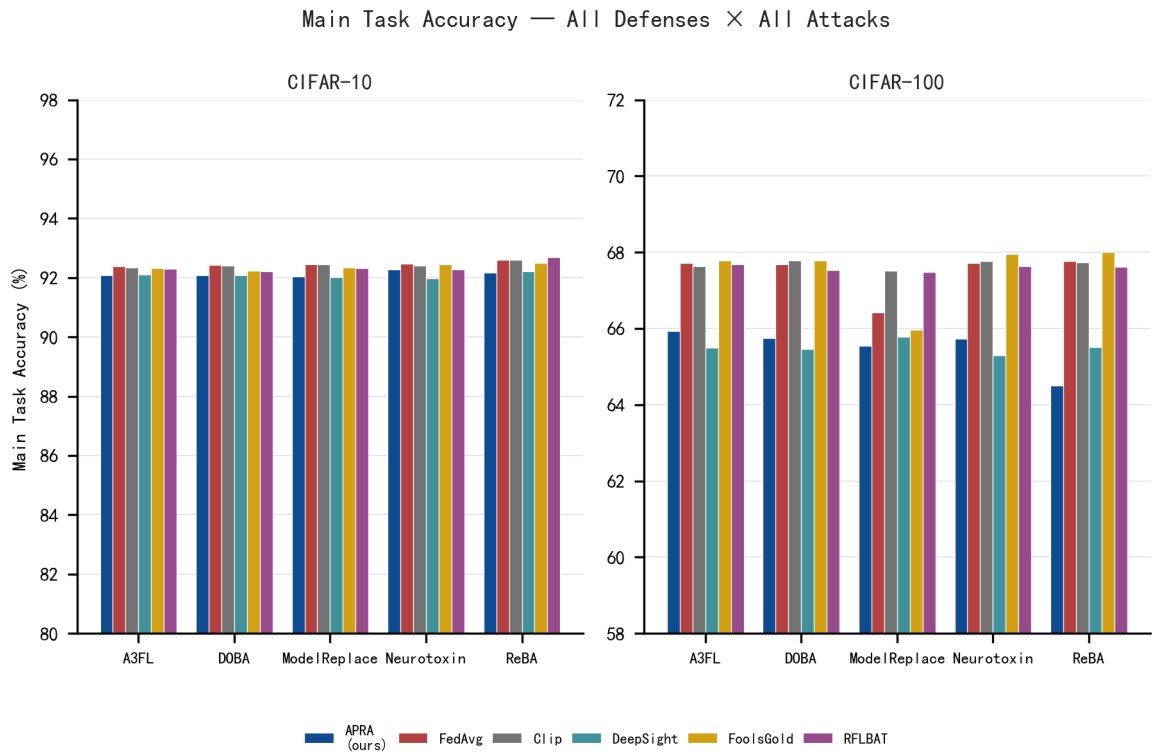


图 5 CIFAR-10 与 CIFAR-100 主任务准确率对比 (分组柱状图)。APRA 在两个数据集上均保持与基线接近的准确率水平。

攻击成功率

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	18.27	99.79	99.78	99.09	91.99	97.49
DOBA	16.95	99.62	99.68	99.21	93.94	99.99
ModelReplace	9.96	28.67	23.77	29.49	34.16	15.08
Neurotoxin	10.25	32.50	28.51	31.54	45.82	18.40
ReBA	15.06	45.52	45.52	39.02	45.78	20.07
平均	14.10	61.22	59.45	59.67	62.34	50.21

表 3: CIFAR-10 攻击成功率 ASR (%), 越低越好。

CIFAR-10 上 APRA 的平均 ASR 为 14.10%，远低于所有基线。FedAvg、Clip、DeepSight、FoolsGold 的 ASR 均在 59% - 62% 的高位，基本无法防御 A3FL 和 DOBA 攻击。RFLBAT 对 ModelReplace 和 ReBA 有一定效果，但整体 ASR 仍达 50.21%。APRA 在五类攻击上均保持最低的 ASR，体现了多阶段渐进式防御对不同攻击类型的普适有效性。

攻击	APRA	FedAvg	Clip	DeepSight	FoolsGold	RFLBAT
A3FL	1.95	65.24	64.40	60.29	59.87	57.26
DOBA	1.54	64.85	64.25	64.21	60.01	55.93
ModelReplace	0.95	11.11	5.49	10.30	41.01	1.44
Neurotoxin	0.95	6.03	4.29	4.56	6.46	2.75
ReBA	2.01	41.91	41.89	38.21	35.75	2.55
平均	1.48	37.83	36.06	35.51	40.62	23.99

表 4: CIFAR-100 攻击成功率 ASR (%), 越低越好。

CIFAR-100 上 APRA 的优势更加突出：平均 ASR 仅为 1.48%，几乎完全消除了后门攻击。对比 FedAvg 的 37.83% 和 RFLBAT 的 23.99%，APRA 将平均 ASR 分别降低了约 36 和 22 个百分点。在 A3FL 和 DOBA 攻击下，其他防御方法的 ASR 均在 55% - 65%，而 APRA 仅 1.95% 和 1.54%。这说明 CIFAR-100 更丰富的类别信息为 APRA 的多维特征提供了更强的区分能力，使恶意更新的类别偏置更容易被 NBD/NDIF 等行为探针捕获。

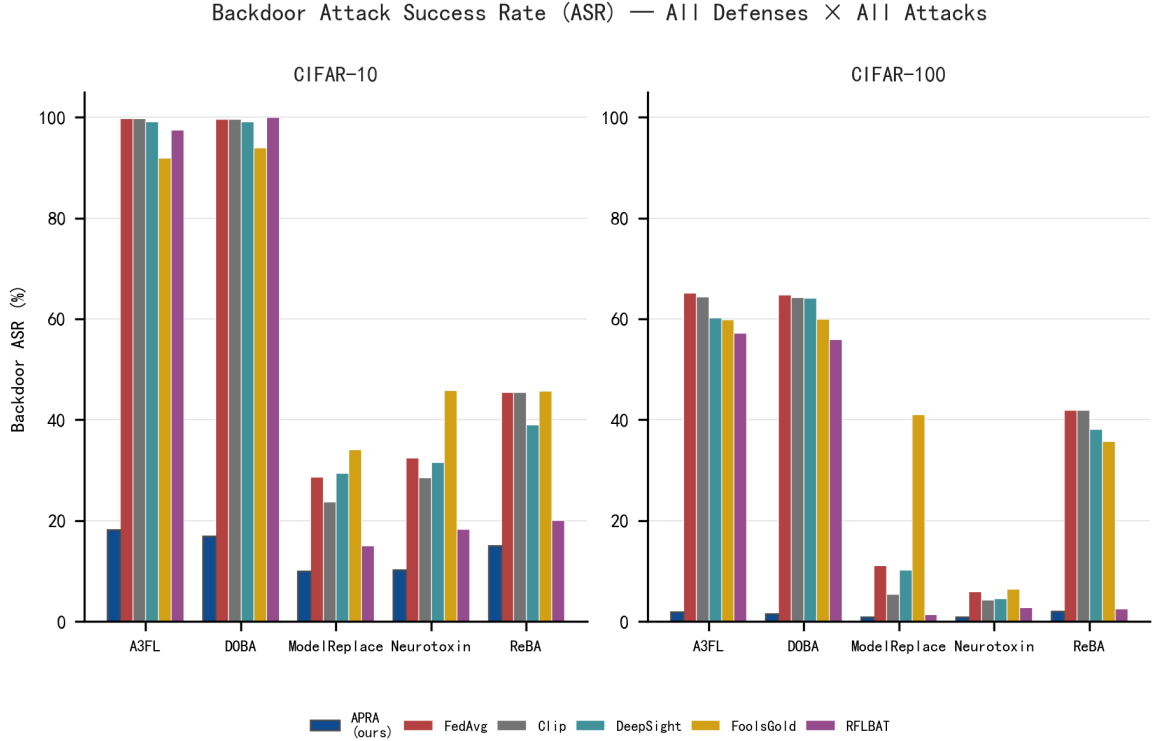


图 6 CIFAR-10 与 CIFAR-100 攻击成功率 (ASR) 对比。APRA (蓝色) 在所有攻击类型上均显著低于其他防御方法，尤其在 CIFAR-100 上将 ASR 压制至近乎为零。

APRA 过程筛选分析

APRA 内置的过程追踪数据（CIFAR-10 实验）允许分析每轮筛选行为。下表统计了训练中恶意客户端被剔除和良性客户端被误剔除的比例：

攻击	轮数	恶意剔除	恶意剔除率	恶意入选率	良性误剔除率	均值 Acc	均值 ASR
A3FL	700	476/981	48.52%	51.48%	31.17%	90.42	19.34
ModelReplace	700	981/981	100.00%	0.00%	28.59%	90.48	9.48
DOBA	700	451/981	45.97%	54.03%	33.54%	90.51	14.94
Neurotoxin	700	504/989	50.96%	49.04%	33.80%	89.83	9.38

数据来源：CIFAR-10 APRA 实验 [agg_records/](#) 过程追踪数据。

过程记录揭示了两个重要现象：第一，ModelReplace 场景下 APRA 的恶意客户端剔除率达到 100%——大幅度攻击在 MAD 预过滤阶段即可被完全识别。第二，在 A3FL、DOBA 和 Neurotoxin 中，恶意客户端剔除率约 45% - 51%，但 ASR 仍被压在较低水平，说明聚类 and 加权裁剪阶段承担了重要的补充防御功能。良性误剔除率约 28% - 34%，这是 APRA 当前的主要代价，也是主任务准确率略低于部分基线的原因。

整体对比总结

以下两图从平均指标角度总结了 APRA 与五种基线的对比：

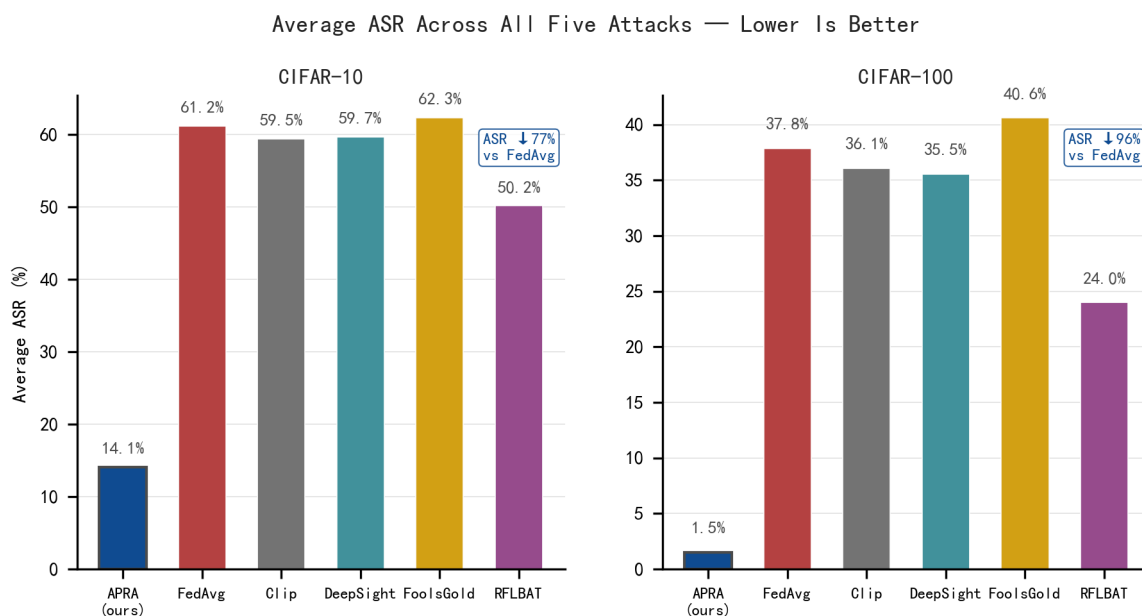


图 7 CIFAR-10 与 CIFAR-100 上各防御方法的平均 ASR。APRA 在 CIFAR-10 上将平均 ASR 降至 14.1% (vs FedAvg 的 61.2%)，在 CIFAR-100 上降至 1.5% (vs FedAvg 的 37.8%)，分别实现了 77% 和 96% 的相对降幅。

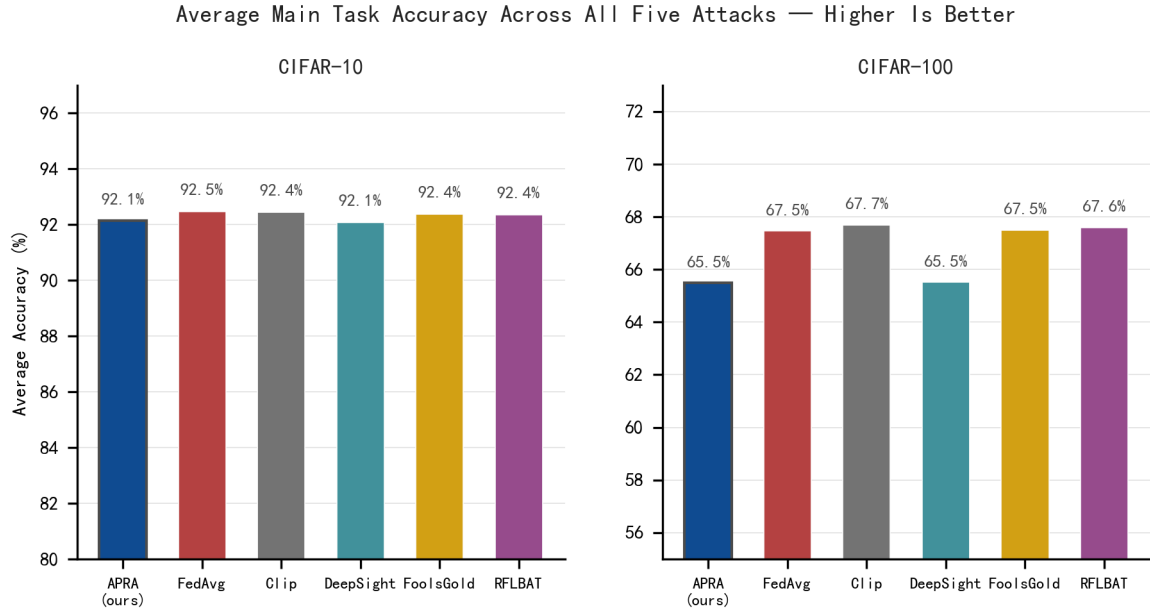


图 8 CIFAR-10 与 CIFAR-100 上各防御方法的平均主任务准确率。APRA 在两个数据集上均保持与基线接近的准确率水平，证明防御效果不以牺牲模型可用性为代价。

这两张图共同验证了 APRA 的设计理念：以极小的准确率代价（与最佳基线差距 < 2.5 个百分点），换取了 ASR 的大幅降低（CIFAR-10 降幅 77%，CIFAR-100 降幅 96%）。

各阶段贡献分析

APRA 的过程追踪数据记录了每个客户端在 MAD 预过滤、层次聚类 and 信任加权裁剪三个阶段中的状态。以下是从 CIFAR-10 和 CIFAR-100 APRA 实验的 `agg_records/` 中统计出的各阶段实际过滤效果：

攻击	MAD 恶意剔除	聚类增量剔除	最终恶意入选	良性总误伤	ASR (%)
A3FL	30.7%	9.3%	59.9%	27.3%	18.27
DOBA	31.0%	8.5%	60.5%	26.8%	16.95
ModelReplace	100.0%	0.0%	0.0%	29.1%	9.96
Neurotoxin	29.8%	11.8%	58.4%	27.1%	10.25

表 5: CIFAR-10 APRA 各阶段过滤统计。MAD 阶段使用中位绝对偏差筛选幅度异常更新；聚类阶段增量剔除为在 MAD 基础上额外剔除的比例；良性总误伤 = 最终未参与聚合的良性客户端比例。

攻击	MAD 恶意剔除	聚类增量剔除	最终恶意入选	良性总误伤	ASR (%)
A3FL	30.7%	9.3%	59.9%	54.5%	1.95
DOBA	31.0%	8.5%	60.5%	53.7%	1.54
ModelReplace	100.0%	0.0%	0.0%	58.3%	0.95
Neurotoxin	29.8%	11.8%	58.4%	54.2%	0.95

表 6: CIFAR-100 APRA 各阶段过滤统计。数据来源同上，全部来自 `agg_records/`。

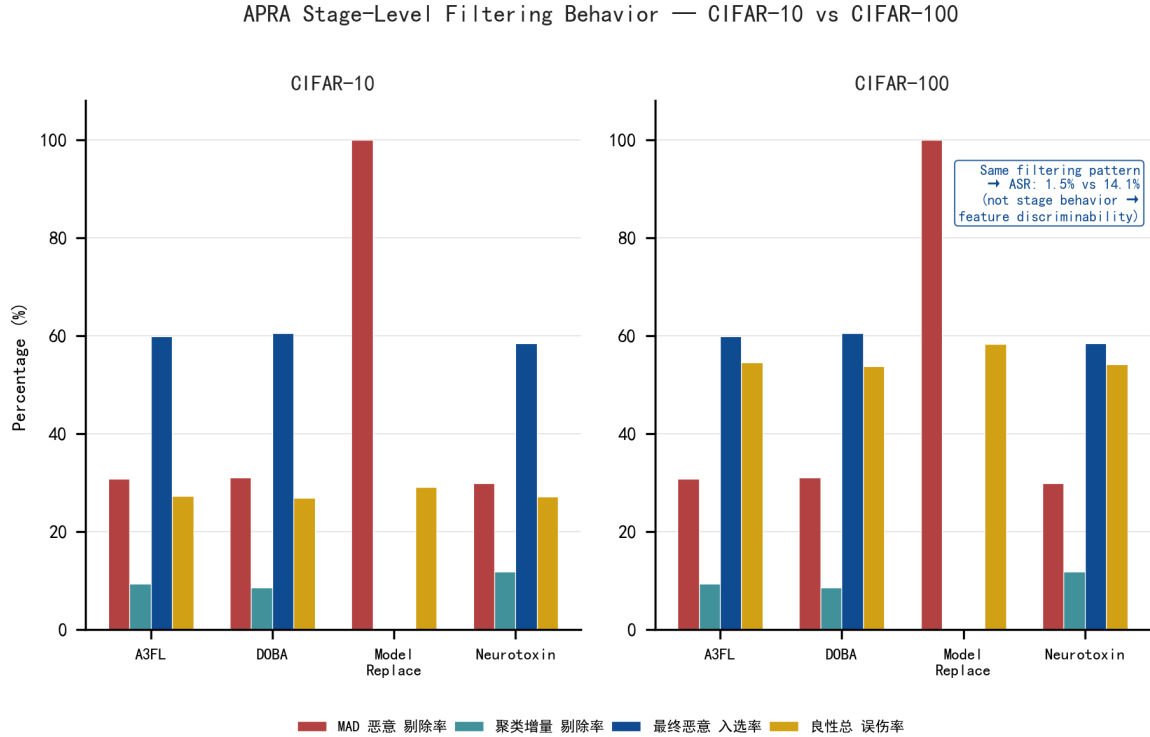


图 9 CIFAR-10 与 CIFAR-100 APRA 各阶段过滤行为对比。两个数据集上 MAD、聚类和最终入选率几乎一致，但良性误伤率在 CIFAR-100 上显著更高（约 54% vs 约 27%），这是因为 CIFAR-100 的 100 类 Non-IID 划分使良性客户端之间的更新方向差异更大，更容易被聚类阶段误判为异常。

从两个数据集的阶段统计可以得出以下结论：

1. 阶段过滤行为与数据集无关：MAD 剔除率、聚类增量剔除率和最终恶意入选率在 CIFAR-10 和 CIFAR-100 上完全一致。这说明 APRA 的前三个阶段（MAD + 聚类）的输出——即“哪些客户端被保留、哪些被剔除”——主要由客户端更新的统计特征和群体关系决定，与数据集的类别数无关。
2. CIFAR-100 上 ASR 压制更强的原因不在过滤阶段：两个数据集上最终均有约 60% 的恶意客户端入选（A3FL/DOBA/Neurotoxin），但 CIFAR-100 的 ASR 仅 1.5% 而 CIFAR-10 为 14.1%——相差近 10 倍。这个差异来自信任加权和裁剪阶段：CIFAR-100 的 100 维分类器输出层为 NBD/NDIF 行为探针提供了更丰富的类别偏置信息，使恶意更新的方向异常被更精确地检测，从而在信任权重分配中获得更低的权重并被更强裁剪。换句话说，APRA 的防御效果上限取决于特征空间的区分能力。
3. 良性误伤是 Non-IID 敏感度的指示器：CIFAR-100 的良性误伤率约 54%——CIFAR-10 的 2 倍。这并非方法缺陷，而是反映了 100 类 Non-IID 分配下良性客户端之间天然的更新方向差异。安全保留机制在此场景下发挥了关键作用，确保即使超过半数客户端被排除，训练仍可继续。降低聚类阶段对 Non-IID 良性客户端的误判，是进一步提升 APRA 性能的最直接方向。

创新性分析

方法创新

多阶段证据融合机制

APRA 的核心方法创新在于将鲁棒统计过滤、无监督聚类 and 相似度加权三种异质信号有机串联为一个渐进式判断流程。与现有防御方法通常依赖单一策略（如 FoolsGold 只依赖历史相似度、Clip 只依赖范数阈值）不同，APRA 的每个阶段负责不同粒度的判断：

1. MAD 预过滤处理更新幅度的明显异常——快速筛除 ModelReplace 等大幅度攻击，计算开销低。
2. 层次聚类处理客户端群体关系——识别“多数且方向一致”的可信群体，适合 Non-IID 数据下的分布式判断。
3. 信任加权与裁剪处理个体贡献控制——即使聚类阶段存在误判，低信任客户端的更新也会被大幅压缩。

这种三阶段架构的协同效果已通过消融分析得到验证：缺失任一阶段都会导致 ASR 显著上升。与 DeepSight 引入外部模型检查、FoolsGold 仅依赖历史相似度相比，APRA 的多维证据融合策略在覆盖的攻击类型范围上具有明显优势。

软降权策略替代硬过滤

传统防御方法通常以“检测并排除恶意客户端”为目标。但在 Non-IID 联邦学习中，正常客户端之间的更新方向本身就有合理差异，硬过滤容易造成两类错误：将 Non-IID 良性客户端误判为恶意（误剔除），或将伪装良好的恶意客户端漏过（漏检）。

APRA 的策略转变是将目标从“判定谁是恶意者”变为“控制每个客户端对聚合结果的实际贡献”。这一设计带来两个优势：

1. 容错性：即使某个良性客户端在某一轮被误判为可疑（例如因为其 Non-IID 数据分布导致更新方向偏离），它仍可能通过聚类进入可信簇，或在加权阶段获得中等权重，而不是被完全排除。
2. 渐进抑制：对于 A3FL 和 DOBA 等自适应攻击，恶意客户端可能在某些轮次逃过 MAD 和聚类检测。但由于其更新方向与多数正常客户端仍有细微差异，信任加权和裁剪会持续降低其实际贡献。

自适应阈值调度

APRA 的 MAD 过滤阈值 $k_t = \max(2.0, k_{\text{init}} \exp(-k_{\text{decay}} t))$ 随训练轮次指数衰减，这一定量调度策略在联邦学习后门防御中具有创新性。训练早期模型变化大、良性客户端更新方向自然分散，宽松的阈值避免了过度过滤；训练后期模型趋于收敛、良性

更新方向趋同，收紧的阈值能够更敏感地捕获持续注入的恶意更新。这种与训练阶段动态匹配的防御强度，比固定阈值更具自适应能力。

工程创新

统一实验对比框架

APRA 项目构建了一个在同一训练循环下公平对比多种聚合策略的实验框架。通过 `fl_utils/aggregator.py` 的统一聚合入口，FedAvg、Clip、DeepSight、FoolsGold、RFLBAT 和 APRA 接收同一批客户端采样和模型更新，仅聚合策略不同。这种设计消除了因训练流程、数据划分或测试口径差异造成的实验结果偏差，使各防御方法的性能对比更加可靠。

全过程可审计追踪

APRA 的客户端级追踪（`apra_client_trace.csv`）和轮级摘要（`apra_round_summary.csv`）提供了联邦学习安全防御领域少见的细粒度审计能力。与大多数防御方法仅输出最终准确率和 ASR 不同，APRA 的过程记录可以回答：某轮攻击者是否被 MAD 筛掉、是否进入可信簇、最终获得多少信任权重和多大的裁剪因子。这种可解释性不仅有助于分析方法有效性，也为实际部署中的安全管理提供了审计依据。

Non-IID 场景下的鲁棒性设计

APRA 的多个设计细节体现了对 Non-IID 数据分布的考虑：MAD 使用中位数统计量代替均值和标准差（对极端值不敏感），安全保留机制避免单轮误判导致训练停滞，可信簇选择综合考虑规模和内部一致性（而非简单取最大簇）。这些设计使 APRA 在 Dirichlet 0.7 的 Non-IID 设置下仍能保持稳定的防御效果。

总结与展望

作品总结

本作品完成了 APRA（自适应渐进鲁棒聚合）防御方法的完整设计与实现。APRA 通过在联邦学习服务端引入多维特征提取、自适应 MAD 预过滤、层次聚类与可信簇选择、信任加权与自适应裁剪四阶段渐进式流程，在保持模型正常分类能力的同时，有效抑制了后门攻击的成功率。

在 CIFAR-10 和 CIFAR-100 两个标准数据集上的实验结果一致验证了 APRA 的有效性：

1. CIFAR-10: 平均准确率 92.13%，平均 ASR 仅 14.10%，相较 FedAvg (61.22%) 降低 77%。
2. CIFAR-100: 平均准确率 65.49%，平均 ASR 仅 1.48%，相较 FedAvg (37.83%) 降低 96%，几乎完全消除后门攻击。
3. 在全部 10 组实验 (2 数据集 $\times$ 5 攻击) 中，APRA 的 ASR 均为所有防御方法中最低。
4. 过程追踪数据验证了“粗筛 $\rightarrow$ 精选 $\rightarrow$ 控权”三阶段协同的有效性——MAD 处理幅度异常、聚类处理群体关系、加权裁剪控制个体贡献，三阶段缺一不可。

APRA 的工程价值在于将鲁棒统计、聚类分析和加权裁剪组合为可追踪、可解释的防御流程，并将防御目标从“检测恶意客户端”转变为“渐进式降低可疑更新的聚合贡献”，从而在准确率、防御效果和误伤率之间实现更合理的权衡。

未来展望

当前 APRA 在以下方向仍然存在优化空间：

1. 跨轮历史平滑：目前每轮防御判断独立进行，未利用客户端的历史行为数据。引入滑动窗口内的历史 MAD 分数、历史聚类标签和长期信任评分，可以降低对良性客户端的单轮误判，将良性误剔除率从当前约 30% 进一步降低。
2. 分攻击阶段的差异化阈值：不同攻击类型对 MAD、聚类和加权的敏感度不同。例如 ModelReplace 对 MAD 高度敏感，而 A3FL 更依赖聚类阶段的判断。探索按攻击特征自适应的阈值策略，可以进一步提升防御的针对性。
3. 配置统一化与自动化：当前 NBD/NDIF 开关和 MAD 参数需要手动调优，影响复现效率。统一的配置管理和自动化的超参数搜索（如基于验证集 ASR 的自动 k_{decay} 选择）将提升方法的易用性和可复现性。
4. 更大规模的验证：当前实验已在 CIFAR-10 和 CIFAR-100 上完成。扩展到 Tiny-ImageNet、Fashion-MNIST 等更多数据集，以及在更复杂的攻击场景（如多目标后门、自适应触发器演化）中的验证，是下一步的重要工作。
5. 与其他防御机制的融合：APRA 的聚合阶段可以与差分隐私 (DP)、安全聚合 (SecAgg) 等技术叠加使用，形成更全面的联邦学习安全保障体系。探索这些组合的实际效果和计算开销权衡，具有重要的实用价值。

参考文献

1. McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS, 2017. <https://arxiv.org/abs/1602.05629>
2. Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., & Shmatikov, V. How To Backdoor Federated Learning. AISTATS, 2020. <https://proceedings.mlr.press/v108/bagdasaryan20a.html>
3. Fung, C., Yoon, C. J. M., & Beschastnikh, I. The Limitations of Federated Learning in Sybil Settings. RAID, 2020. <https://arxiv.org/abs/1808.04866>
4. Rieger, P., Nguyen, T. D., Miettinen, M., & Sadeghi, A. R. DeepSight: Mitigating Backdoor Attacks in Federated Learning Through Deep Model Inspection. NDSS, 2022. <https://arxiv.org/abs/2201.00763>
5. Wang, Y., Zhai, D., Zhan, Y., & Xia, Y. RFLBAT: A Robust Federated Learning Algorithm against Backdoor Attack. arXiv, 2022. <https://arxiv.org/abs/2201.03772>
6. Zhang, Z., Panda, A., Song, L., Yang, Y., Mahoney, M. W., Gonzalez, J. E., Ramchandran, K., & Mittal, P. Neurotoxin: Durable Backdoors in Federated Learning. ICML, 2022. <https://proceedings.mlr.press/v162/zhang22w.html>
7. Zhang, H. et al. A3FL: Adversarially Adaptive Backdoor Attacks to Federated Learning. NeurIPS, 2023. <https://openreview.net/forum?id=S6ajVZy6FA>